

第一部分：理论篇

1.1 什么是进程？

进程（Process）就是正在运行的程序。

程序是静态的，存储在硬盘上的一堆代码文件。当你双击运行它，系统会把它加载到内存中执行，这时它就变成了进程。

生活化类比：菜谱与做菜

概念	类比	说明
程序	菜谱	写在纸上的步骤，放着不动什么都不会发生
进程	做菜过程	厨师按照菜谱实际操作，占用厨房资源
CPU	厨师	真正干活的人
内存	厨房台面	放置食材和工具的空间

一个程序可以同时运行多次，产生多个进程。就像同一份菜谱，可以同时有多个厨师在做。

1.2 进程的基本属性

每个进程都有自己的"身份证"和属性：

属性	说明	类比
PID	进程ID，唯一标识符	身份证号
PPID	父进程ID	你爸是谁
UID	运行该进程的用户ID	这是谁的进程
状态	运行、睡眠、停止等	正在工作、休息、暂停
优先级	获取CPU时间的优先程度	VIP等级
内存占用	使用多少内存	占了多大地方
CPU占用	使用多少CPU资源	干活多卖力

1.3 进程的状态

进程在生命周期中会经历不同的状态：

状态	符号	说明	类比
运行态	R (Running)	正在CPU上执行或等待执行	正在做菜或排队等灶台
睡眠态	S (Sleeping)	等待某个事件，可被唤醒	等食材送来，随时能开工
深度睡眠	D (Disk sleep)	不可中断的睡眠，通常等待I/O	等烤箱，必须等到响
停止态	T (Stopped)	被暂停，可以恢复	被叫去接电话，回来继续
僵尸态	Z (Zombie)	已结束但未被父进程回收	菜做完了但没人收拾

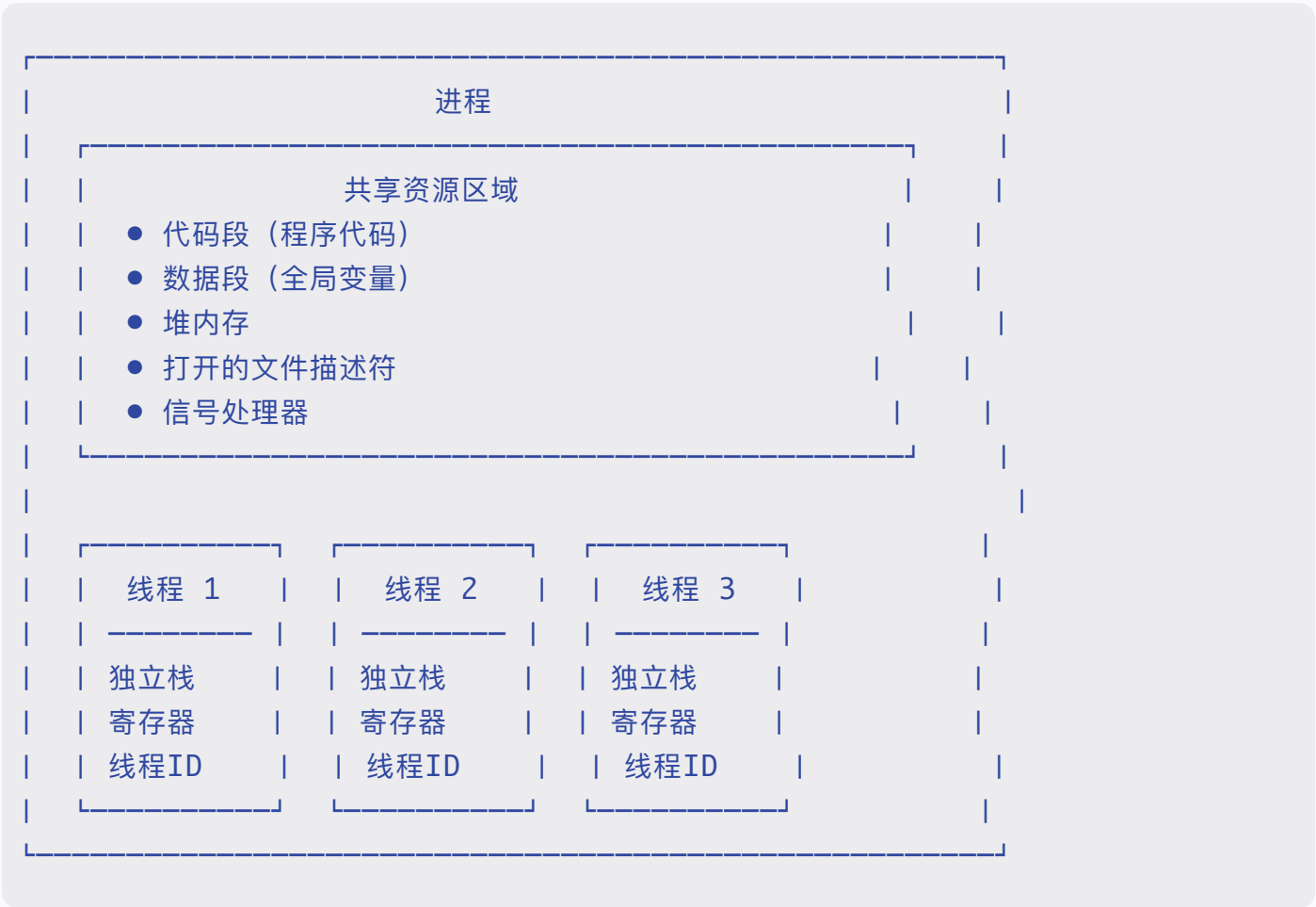
1.4 什么是线程

线程是进程内的执行单元，是 CPU 调度的基本单位。一个进程可以包含多个线程。

线程的特点：

- 同一进程的线程共享内存空间和资源
- 线程有自己独立的栈空间和寄存器
- 线程之间切换开销比进程小
- 一个线程崩溃可能导致整个进程崩溃

1.5 进程与线程的关系



1.6 形象比喻

把进程比作一个工厂，线程就是工厂里的工人。工厂有自己的厂房、设备、原材料（资源），工人们共享这些资源来完成工作。每个工人有自己的工作台（栈空间）。工厂之间是独立的，一个工厂倒闭不影响其他工厂；但同一工厂的工人出了问题，可能会影响整个工厂的运转。

对比项	进程	线程
定义	独立的程序执行实例	进程内的执行单元
资源	有独立的内存空间	共享进程的内存空间
开销	创建和切换开销大	创建和切换开销小
通信	需要专门的进程间通信机制	可直接共享数据
类比	不同的餐厅	同一餐厅的不同厨师

简单记忆： 进程是资源分配的基本单位， 线程是CPU调度的基本单位。

1.6 前台进程与后台进程

类型	特点	示例
前台进程	占用终端，可接收键盘输入	直接运行 <code>vim file.txt</code>
后台进程	不占用终端，在后台静默运行	运行 <code>./script.sh &</code>

后台运行的好处：可以同时做其他事情，终端不会被占用。

1.7 守护进程（Daemon）

守护进程是一种特殊的后台进程，它在系统启动时自动运行，不与任何终端关联，默默提供服务。

常见的守护进程：

进程名	作用
sshd	提供SSH远程登录服务
httpd/nginx	提供Web服务
mysqld	提供数据库服务
crond	提供定时任务服务
systemd	系统和服务管理器（1号进程）

守护进程通常以 **d** 结尾，d 代表 daemon。

1.8 信号（Signal）

信号是进程间通信的一种方式，用于通知进程发生了某个事件。

常用信号：

信号编号	信号名	作用	键盘快捷键
1	SIGHUP	挂起，通常让服务重新加载配置	—
2	SIGINT	中断，相当于 Ctrl+C	Ctrl+C
9	SIGKILL	强制杀死，不可被忽略	—
15	SIGTERM	终止，默认信号，可被捕获	—
18	SIGCONT	继续运行被停止的进程	—
19	SIGSTOP	停止进程，不可被忽略	—
20	SIGTSTP	暂停进程	Ctrl+Z

SIGTERM vs SIGKILL：

SIGTERM 是"请你退出", 进程可以做清理工作后优雅退出;

SIGKILL 是"立刻去死", 进程没有机会做任何事情。

优先使用 SIGTERM, 实在不行再用 SIGKILL。

第二部分：实践篇

2.1 查看进程

2.1.1 ps 命令：进程快照

`ps` 命令显示当前时刻的进程状态，就像拍一张照片。

基础用法：

```
# 查看当前终端的进程
ps

# 输出示例：
#   PID TTY          TIME CMD
#  1234 pts/0        00:00:00 bash
#  5678 pts/0        00:00:00 ps
```

常用参数组合：

```
# 查看所有进程（BSD风格，常用）
ps aux

# 查看所有进程（Unix风格）
ps -ef

# 两者功能相似，显示格式略有不同
```

`ps aux` 输出详解：

```
ps aux
```

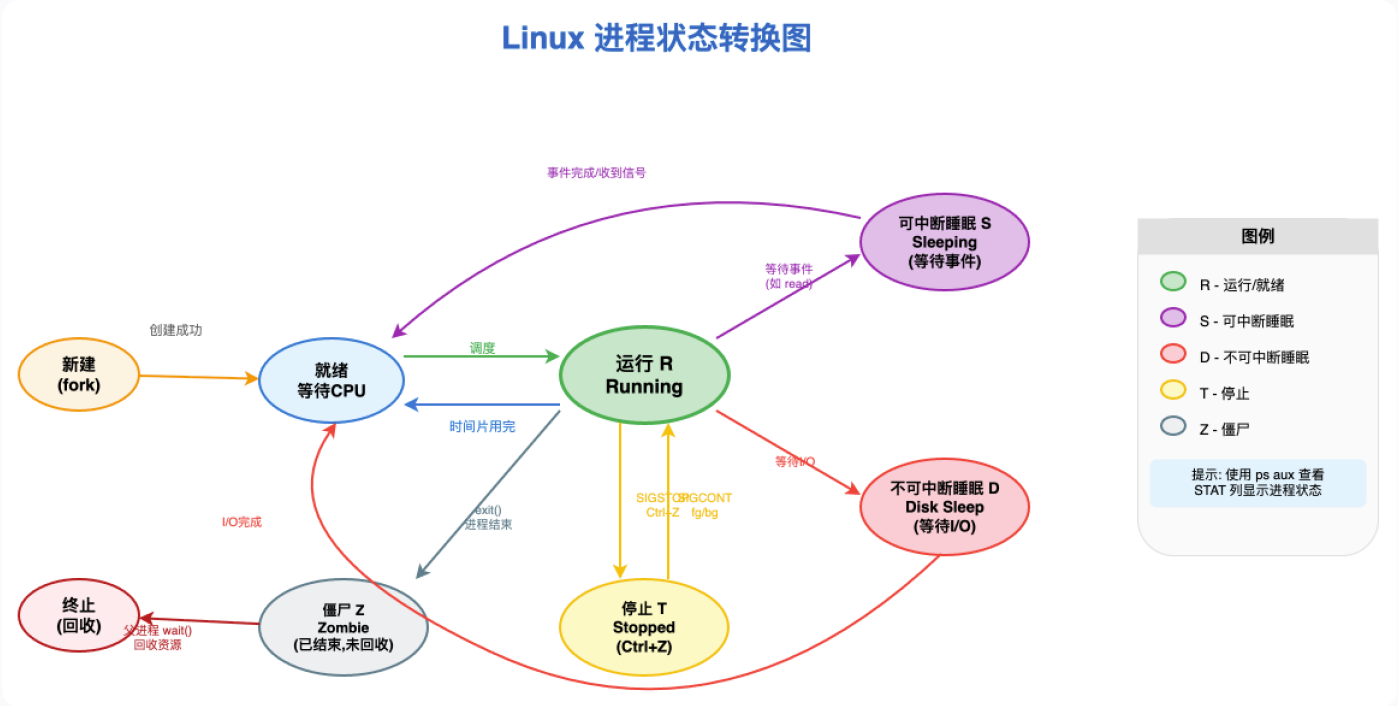
```
# 输出示例:
```

```
# USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME
COMMAND
# root           1  0.0  0.3 225848  9632 ?        Ss   09:00   0:03
/usr/lib/systemd/systemd
# root           2  0.0  0.0      0      0 ?        S    09:00   0:00
[kthreadd]
```

列名	含义
USER	进程所有者
PID	进程ID
%CPU	CPU使用率
%MEM	内存使用率
VSZ	虚拟内存大小(KB)
RSS	实际内存大小(KB)
TTY	终端，? 表示无终端
STAT	进程状态
START	启动时间
TIME	累计CPU时间
COMMAND	命令名称

STAT 状态码详解:

状态码	含义
R	运行中
S	可中断睡眠
D	不可中断睡眠
T	已停止
Z	僵尸进程
s	会话首进程
l	多线程
+	前台进程
<	高优先级
N	低优先级



实用技巧：

```
# 查找特定进程
ps aux | grep nginx

# 按 CPU 使用率排序（降序）
ps aux --sort=-%cpu | head -10

# 按内存使用率排序（降序）
ps aux --sort=-%mem | head -10

# 查看进程树
ps -ef --forest

# 只显示特定列
ps -eo pid,ppid,user,%cpu,%mem,cmd
```

2.1.2 top 命令：实时监控

`top` 是实时的进程监控工具，每隔几秒刷新一次，就像看实时视频。

```
# 启动 top
top
```

`top` 界面详解：

```
top - 14:30:00 up 5 days, 3:45, 2 users, load average: 0.15, 0.10, 0.05
Tasks: 128 total, 1 running, 127 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.3 us, 1.0 sy, 0.0 ni, 96.5 id, 0.2 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3936.0 total, 1234.5 free, 1456.2 used, 1245.3 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 2123.4 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+
COMMAND
 1234 root      20   0 225848   9632   7168 S   0.3   0.2   0:03.45
systemd
```

第一行解读：

项目	示例值	含义
当前时间	14:30:00	系统当前时间
up	5 days, 3:45	系统运行时长
users	2 users	登录用户数
load average	0.15, 0.10, 0.05	1/5/15分钟平均负载

负载（Load Average）理解：

负载表示等待 CPU 处理的进程数量。单核 CPU 时，负载 1.0 表示刚好满载，大于 1.0 表示过载。多核 CPU 时，核心数等于满载值，例如 4 核 CPU 负载 4.0 表示满载。

CPU 行解读：

项目	含义
us (user)	用户空间进程占用
sy (system)	内核空间占用
ni (nice)	改变过优先级的进程占用
id (idle)	空闲
wa (wait)	等待I/O

top 交互命令：

按键	功能
q	退出
h	帮助
P	按CPU排序（默认）
M	按内存排序
T	按运行时间排序
k	杀死进程
r	修改nice值
l	显示每个CPU核心
c	显示完整命令行
f	选择显示的字段

常用参数：

指定刷新闻隔（秒）

```
top -d 2
```

只显示特定用户的进程

```
top -u nginx
```

只显示特定进程

```
top -p 1234,5678
```

批处理模式（可用于脚本）

```
top -bn1
```

2.1.3 htop：更友好的 top

htop 是 top 的增强版，界面更美观，操作更友好。

```
# 安装
yum install -y htop      # CentOS/RHEL
apt install -y htop      # Ubuntu/Debian

# 运行
htop
```

htop 优势：

特性	top	htop
界面	纯文本	彩色图形化
滚动	不支持	支持鼠标滚动
操作	记命令	底部有菜单提示
进程树	需参数	F5 一键切换
搜索	不直观	F3 直接搜索
杀进程	输入 k 和 PID	F9 选择后杀死

2.1.4 pstree：查看进程树

```
# 安装
yum install -y psmisc      # CentOS/RHEL

# 查看进程树
pstree

# 显示 PID
pstree -p

# 显示特定进程的子树
pstree -p 1234
```

```
# 高亮当前进程
pstree -h
```

输出示例：

```
systemd└─NetworkManager---2*[{NetworkManager}]
      ├─sshd---sshd---sshd---bash---pstree
      ├─crond
      └─rsyslogd---2*[{rsyslogd}]
```

2.1.5 pgrep：搜索进程

```
# 按名称查找进程 PID
pgrep nginx

# 显示进程名和 PID
pgrep -l nginx

# 显示完整命令行
pgrep -a nginx

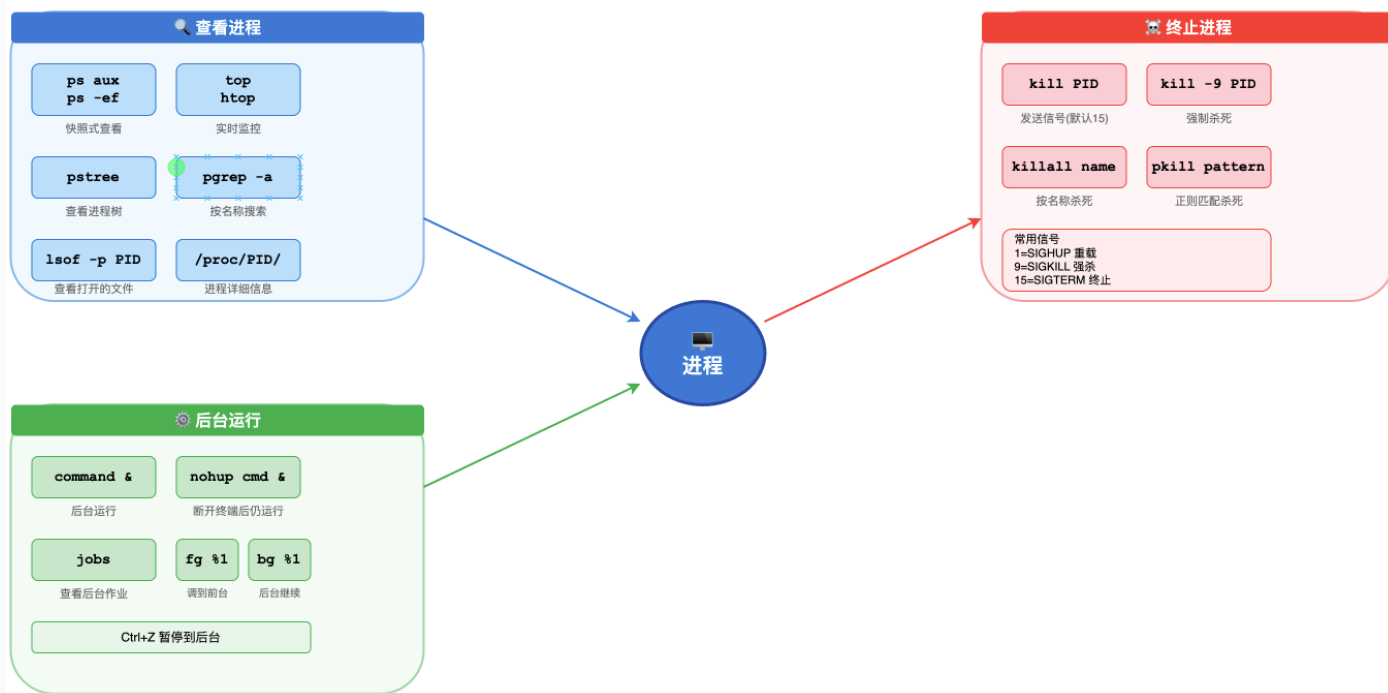
# 查找特定用户的进程
pgrep -u root sshd

# 精确匹配进程名
pgrep -x nginx
```

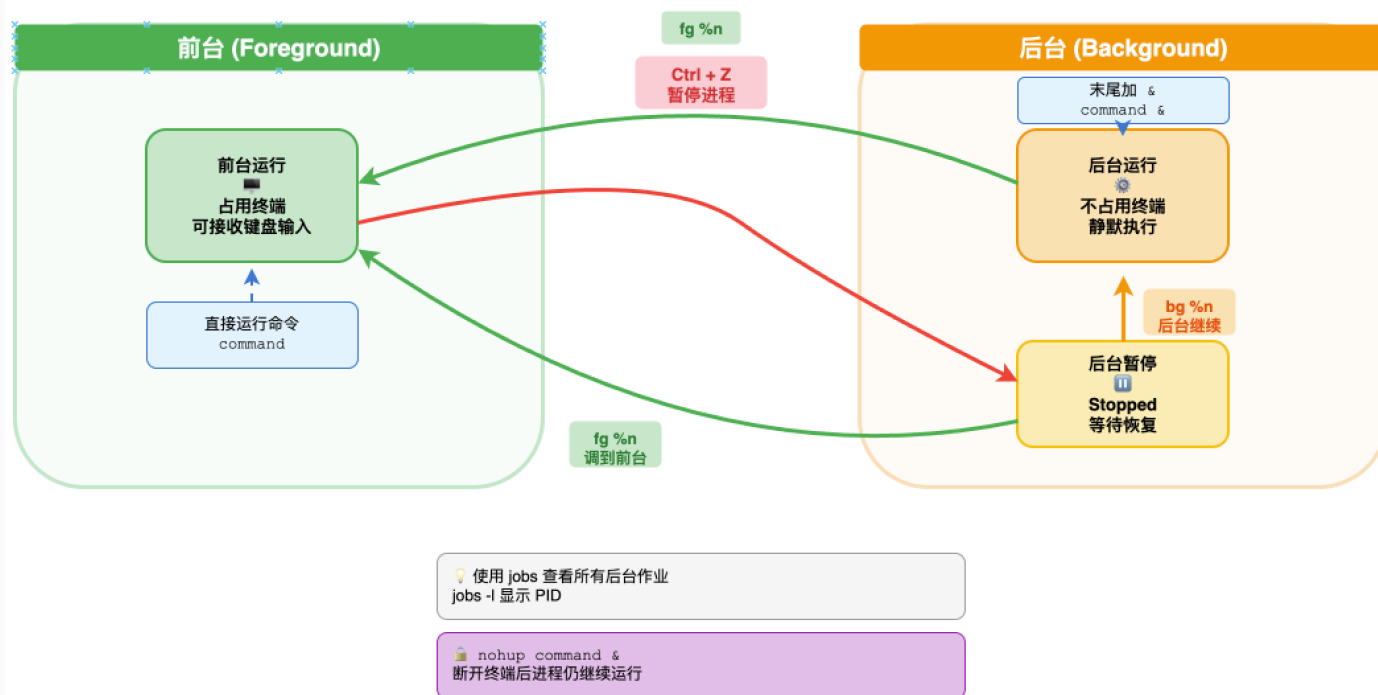
2.2 管理进程

2.2.1 前台与后台切换

Linux 进程管理命令速查



前台与后台进程切换



在后台运行命令：

```
# 命令末尾加 &
./long_running_script.sh &

# 输出示例：
# [1] 12345    ← [作业号] PID
```

查看后台作业：

```
jobs

# 输出示例：
# [1]+  Running    ./long_running_script.sh &
# [2]-  Stopped    vim file.txt
```

前后台切换：

```
# 将前台进程放到后台（先暂停）
# 按 Ctrl+Z
# 输出：[1]+  Stopped    ./script.sh

# 让停止的后台作业继续运行
bg %1

# 将后台作业调到前台
fg %1

# %1 是作业号，可以省略 % 写成 fg 1
```

实用示例：

场景：运行了一个很长的命令，想临时干别的事

1. 按 Ctrl+Z 暂停

^Z

[1]+ Stopped ./backup.sh

2. 让它在后台继续运行

bg %1

[1]+ ./backup.sh &

3. 现在可以做其他事情了

ls -la

2.2.2 nohup：断开终端后继续运行

问题：SSH 断开后，后台进程也会被终止。

解决：使用 `nohup`，让进程不受挂断信号影响。

基本用法

nohup ./long_script.sh &

输出默认保存到 nohup.out

指定输出文件

nohup ./script.sh > output.log 2>&1 &

2>&1 表示将标准错误也重定向到标准输出

完整示例：

```
# 运行一个长时间任务
nohup ./data_process.sh > /var/log/process.log 2>&1 &

# 记录 PID
echo $! > /var/run/process.pid

# 之后可以安全断开 SSH
exit

# 重新登录后查看进程
ps aux | grep data_process
```

2.2.3 终止进程

kill 命令：

```
# 默认发送 SIGTERM (15)
kill 1234

# 指定信号
kill -9 1234      # SIGKILL, 强制杀死
kill -15 1234     # SIGTERM, 优雅终止
kill -1 1234      # SIGHUP, 重新加载配置

# 使用信号名称
kill -SIGTERM 1234
kill -TERM 1234
```

killall 命令：

按进程名杀死所有匹配的进程

```
killall nginx
```

强制杀死

```
killall -9 nginx
```

只杀死属于特定用户的进程

```
killall -u nginx nginx
```

交互式确认

```
killall -i nginx
```

pkill 命令：

按进程名杀死（支持正则）

```
pkill nginx
```

按用户杀死

```
pkill -u www
```

按终端杀死

```
pkill -t pts/1
```

强制杀死

```
pkill -9 nginx
```

正确的杀进程顺序：

第一步：优雅终止

```
kill 1234
```

等待几秒

```
sleep 5
```

第二步：如果还活着，强制杀死

```
kill -9 1234
```

2.3 实验练习

实验一：进程查看与分析

目标：熟练使用 ps、top 查看和分析进程

步骤：

```
# 1. 查看系统中所有进程数量
ps aux | wc -l

# 2. 找出 CPU 占用最高的 5 个进程
ps aux --sort=-%cpu | head -6

# 3. 找出内存占用最高的 5 个进程
ps aux --sort=-%mem | head -6

# 4. 查看当前用户的所有进程
ps -u $(whoami)

# 5. 使用 top 观察系统
top

# 在 top 中练习：
# - 按 P 按 CPU 排序
# - 按 M 按内存排序
# - 按 1 查看各 CPU 核心
# - 按 c 显示完整命令
# - 按 q 退出
```

实验二：前后台进程管理

目标：掌握前后台进程的切换和管理

步骤：

```
# 1. 创建测试脚本
cat > /tmp/test_bg.sh << 'EOF'
#!/bin/bash
for i in {1..100}; do
```

```
    echo "Running... $i"
    sleep 2
done
EOF
chmod +x /tmp/test_bg.sh

# 2. 前台运行
/tmp/test_bg.sh
# 观察输出，然后按 Ctrl+Z 暂停

# 3. 查看作业
jobs

# 4. 放到后台继续运行
bg %1

# 5. 再次查看
jobs

# 6. 调回前台
fg %1

# 7. 按 Ctrl+C 终止
```

实验三：nohup 实验

目标：让进程在终端断开后继续运行

步骤：

```
# 1. 创建长时间运行的脚本
cat > /tmp/long_task.sh << 'EOF'
#!/bin/bash
for i in {1..3600}; do
    echo "$(date): tick $i" >> /tmp/long_task.log
    sleep 1
done
EOF
chmod +x /tmp/long_task.sh
```

```
# 2. 使用 nohup 在后台运行
nohup /tmp/long_task.sh &

# 3. 记录 PID
echo "PID: $!"

# 4. 验证进程在运行
ps aux | grep long_task

# 5. 查看输出
tail -f /tmp/long_task.log

# 6. 断开 SSH 重新连接, 验证进程还在
exit
# 重新连接后
ps aux | grep long_task

# 7. 清理
pkill -f long_task.sh
rm -f /tmp/long_task.sh /tmp/long_task.log /tmp/nohup.out
```

实验四：进程终止实验

目标：掌握各种终止进程的方法

步骤：

```
# 1. 启动几个测试进程
sleep 1000 &
sleep 2000 &
sleep 3000 &

# 2. 查看
jobs
ps aux | grep sleep

# 3. 使用 kill 终止第一个
kill %1
# 或 kill <PID>
```

```
# 4. 使用 killall 终止所有 sleep
```

```
killall sleep
```

```
# 5. 验证
```

```
ps aux | grep sleep
```

实验五：僵尸进程实验

目标：理解僵尸进程的产生和处理

步骤：

```
# 1. 创建产生僵尸进程的脚本
```

```
cat > /tmp/zombie_creator.sh << 'EOF'
```

```
#!/bin/bash
```

```
# 父进程
```

```
(
```

```
    # 子进程
```

```
    echo "子进程启动, PID: $$"
```

```
    sleep 2
```

```
    echo "子进程结束"
```

```
    exit 0
```

```
) &
```

```
echo "父进程 PID: $$"
```

```
echo "子进程 PID: $!"
```

```
echo "父进程开始睡眠, 不回收子进程..."
```

```
sleep 30
```

```
# 父进程不调用 wait, 子进程会变成僵尸
```

```
EOF
```

```
chmod +x /tmp/zombie_creator.sh
```

```
# 2. 运行脚本
```

```
/tmp/zombie_creator.sh &
```

```
# 3. 等待 5 秒后查看
```

```
sleep 5
```

```
ps aux | grep -E 'Z|zombie'
```

```
# 4. 观察状态列是否有 Z
```

```
# 正常情况下现代 Shell 会自动回收子进程，可能看不到僵尸
```

```
# 5. 清理
```

```
pkill -f zombie_creator
```

```
rm -f /tmp/zombie_creator.sh
```

如何处理僵尸进程：

僵尸进程本身不占用资源，但占用 PID。解决方法是杀死其父进程，让 init/systemd 接管并回收。

```
# 找到僵尸进程的父进程
```

```
ps -eo pid,ppid,stat,cmd | grep Z
```

```
# 杀死父进程
```

```
kill <PPID>
```

2.4 常用命令速查表

场景	命令
查看所有进程	<code>ps aux</code> 或 <code>ps -ef</code>
按名称搜索进程	<code>ps aux grep nginx</code> 或 <code>pgrep -a nginx</code>
实时监控进程	<code>top</code> 或 <code>htop</code>
查看进程树	<code>pstree -p</code>
后台运行	<code>command &</code>
查看后台作业	<code>jobs</code>
前台→后台	<code>Ctrl+Z</code> 然后 <code>bg</code>
后台→前台	<code>fg %1</code>
断开终端后运行	<code>nohup command &</code>
终止进程	<code>kill PID</code>
强制终止	<code>kill -9 PID</code>
按名称终止	<code>killall nginx</code> 或 <code>pkill nginx</code>

第四部分：课后练习

练习一：基础查看（必做）

1. 使用 `ps aux` 找出系统中 CPU 占用最高的 3 个进程
2. 使用 `ps aux` 找出系统中内存占用最高的 3 个进程
3. 使用 `pstree` 查看进程树，找到 `sshd` 的子进程
4. 使用 `top`，切换按 CPU 和内存排序

练习二：后台运行（必做）

1. 编写一个脚本，每秒输出当前时间到文件
2. 使用 `&` 将其放到后台运行
3. 使用 `jobs` 查看作业
4. 使用 `fg` 调到前台，然后 `Ctrl+Z` 暂停
5. 使用 `bg` 让其在后台继续运行
6. 使用 `nohup` 重新运行，断开终端后验证进程是否存活

练习三：进程终止（必做）

1. 启动 3 个 `sleep 1000 &` 后台进程
2. 使用 `kill` 终止其中一个
3. 使用 `killall` 终止剩余的
4. 验证所有进程已终止

练习四：综合实战（选做）

模拟一个运维场景：某服务占用过高 CPU，需要排查并处理。

1. 创建一个 CPU 密集型进程
 2. 使用 `top` 找到它
 3. 最终使用 `kill` 终止它
-

第五部分：课程总结

核心知识点回顾

进程是什么：进程是运行中的程序实例，有自己的 PID、状态、资源占用等属性。

进程状态：主要有运行态（R）、睡眠态（S/D）、停止态（T）、僵尸态（Z），可通过 `ps` 的 STAT 列查看。

查看进程：`ps aux` 看快照，`top/htop` 看实时，`pstree` 看层级关系，`pgrep` 按名搜索。

管理进程：& 后台运行，Ctrl+Z 暂停，bg/fg 切换，nohup 防断开终止，kill/killall/pkill 终止进程。

生产环境建议

排查问题时，先用 top 快速定位异常进程，再用 ps aux 获取详细信息。

终止进程时，优先使用 kill (SIGTERM)，给进程清理机会，实在不行再用 kill -9。

长时间运行的后台任务，务必使用 nohup 或写成 systemd 服务，避免终端断开导致任务中断。

定期检查是否有僵尸进程（Z 状态），过多僵尸进程可能意味着程序 bug。